

Amazon Web Services (AWS)

Java Backend Interview — Short Notes

1. Core Compute

EC2 — Elastic Compute Cloud

- Virtual servers in the cloud with full OS control
- **Instance types:** General (t3, m6), Compute (c6), Memory (r6), GPU (p4), Storage (i3)
- **AMI** — Amazon Machine Image: pre-configured OS + software snapshot used to launch instances
- **Security Groups** — stateful virtual firewall at instance level (inbound/outbound rules)
- **Auto Scaling Group** — automatically add/remove EC2 instances based on demand
- **Placement Groups** — Cluster (low latency), Spread (high availability), Partition (large distributed)

Best Case

- Long-running stateful workloads needing full OS control, custom JVM tuning, persistent DB connections
- Example: Spring Boot monolith with custom JVM flags (-Xmx, GC tuning) hosted on EC2 r6i.large with EBS storage

Pros & Cons

Pros	Cons
Full OS and config control Wide instance variety for every workload Spot instances for up to 90% cost savings Supports any language/runtime	You manage patching, scaling, availability Idle instances still cost money More operational overhead than serverless Scaling is slower than Lambda
■ Cost: Per hour/second by instance type. On-Demand most expensive; Reserved (1-3yr) saves ~40-60%; Spot up to 90% cheaper but can be interrupted. e.g., t3.medium On-Demand ~\$0.04/hr	

Lambda — Serverless Functions

- Run code without provisioning servers — triggered by events
- **Triggers:** S3, API Gateway, SQS, SNS, DynamoDB Streams, EventBridge, CloudWatch
- **Cold start:** First invocation spins up container — adds latency (100ms–1s). Mitigate with Provisioned Concurrency
- **Limits:** 15 min max timeout, 10GB memory, 512MB–10GB /tmp storage, 1000 concurrent executions default
- **Concurrency:** Reserved (guarantee), Provisioned (pre-warmed, eliminates cold start)

Best Case

- Short-lived, event-driven, spiky or unpredictable traffic — no traffic = no cost
- Example: Resize image on S3 upload; process SQS order messages; REST API via API Gateway with moderate traffic

Pros & Cons

Pros	Cons
No server management Auto-scales to zero (pay only for use) Free tier: 1M requests + 400K GB-sec/month Native AWS event integration	Cold start latency 15-min max timeout (not for long jobs) Not for stateful or long-running workloads Debugging and local testing is harder
■ Cost: Per invocation (\$0.20 per 1M) + per GB-second of execution. First 1M requests and 400K GB-sec free per month. Very cost-effective for spiky workloads	

ECS — Elastic Container Service

- Run Docker containers on AWS — managed orchestration without Kubernetes complexity
- **Launch types:** EC2 (you manage nodes) or Fargate (serverless — AWS manages nodes)

- **Task Definition** — blueprint for container (image, CPU, memory, env vars, ports)
- **Service** — maintains desired number of running tasks, integrates with ALB

Best Case

- Containerized microservices without managing EC2 nodes — teams already using Docker
- Example: 5 Spring Boot microservices each in Docker containers, independently deployable and scalable on Fargate

Pros & Cons

Pros	Cons
No cluster management with Fargate Scales per task, not per server Deep AWS integration (IAM, ALB, CloudWatch) Simpler than Kubernetes	Less flexibility than EKS for complex orchestration Fargate slightly more expensive than EC2 for steady load Limited ecosystem compared to Kubernetes

■ Cost: Fargate: per vCPU/hour + per GB memory/hour. e.g., 0.25 vCPU + 0.5GB ~\$0.01/hr per task. EC2 launch type: pay for the EC2 instances regardless of utilization

EKS — Elastic Kubernetes Service

- Managed Kubernetes control plane on AWS
- **Node groups:** Managed (AWS handles patching), Self-managed, or Fargate
- Best for: service mesh (Istio), custom scheduling, complex multi-cluster setups, portability

Best Case

- Large-scale microservices needing advanced orchestration — canary deploys, service mesh, complex networking
- Example: 50+ services with Istio service mesh, custom HPA policies, blue/green deployments across multiple AZs

Pros & Cons

Pros	Cons
Full Kubernetes power and ecosystem Portable across cloud providers Advanced orchestration, service mesh support Large community	High operational complexity Steep learning curve \$0.10/hr per cluster (\$72/month minimum) Overkill for simple containerized apps

■ Cost: \$0.10/hr per EKS cluster + EC2/Fargate node costs. A basic 3-node t3.medium cluster = ~\$150/month minimum

2. Storage

S3 — Simple Storage Service

- Object storage — store any file type, any size, at massive scale
- **Storage classes:** Standard, Intelligent-Tiering, Standard-IA, Glacier (archival), Glacier Deep Archive
- **Presigned URLs** — time-limited URL to allow upload/download without exposing credentials
- **Lifecycle policies** — auto-transition objects between storage classes or delete after N days
- **Versioning** — keep multiple versions of an object; enables recovery from accidental deletes
- **Durability:** 99.99999999% (11 nines) — data replicated across minimum 3 AZs

Best Case

- Storing and serving unstructured data — files, images, backups, logs, static website assets
- Example: Users upload profile pictures → S3 bucket → served via CloudFront CDN to users globally with low latency

Pros & Cons

Pros	Cons
------	------

Virtually unlimited storage 11 nines durability Lifecycle policies reduce cost automatically Native integration with almost every AWS service

Not a filesystem (object store only) Higher latency than EBS for frequent reads Data transfer out costs add up Eventual consistency in some edge cases

■ Cost: Per GB stored (~\$0.023/GB Standard) + per 1000 PUT/GET requests + data transfer out. First 5GB free. Glacier ~\$0.004/GB

EBS — Elastic Block Store

- Block storage attached to EC2 — like a virtual hard disk
- **Types:** gp3 (general SSD), io2 (high IOPS), st1 (throughput HDD), sc1 (cold HDD)
- Attached to one EC2 instance at a time (Multi-Attach available for io2)
- **Snapshots** — point-in-time backups stored in S3; used for disaster recovery

Best Case

- Database storage on EC2 requiring low-latency I/O — OS volumes, database data files
- Example: MySQL on EC2 using EBS gp3 volume for data storage — low latency reads/writes, snapshotted nightly to S3

Pros & Cons

Pros	Cons
Low latency, high IOPS Snapshots for easy backup/restore gp3 allows independent IOPS/throughput tuning Persists independently of EC2 instance	Attached to one instance at a time Charged per GB provisioned (even if unused) Not shared across multiple instances Bound to one AZ

■ Cost: gp3: ~\$0.08/GB/month. io2: ~\$0.125/GB/month + per IOPS. Charged for provisioned size regardless of actual usage

EFS — Elastic File System

- Managed NFS shared filesystem — multiple EC2 instances can mount simultaneously
- Auto-scales storage up and down — no capacity planning needed
- **Performance modes:** General Purpose, Max I/O. **Throughput modes:** Bursting, Provisioned, Elastic

Best Case

- Shared filesystem accessed by multiple EC2 instances at the same time
- Example: Multiple EC2 app servers read/write shared config files or user-uploaded documents from a single EFS mount

Pros & Cons

Pros	Cons
Shared access across many instances Auto-scales — no provisioning needed Multi-AZ by default Good for CMS, shared config, home dirs	Higher cost than EBS (~3x per GB) Higher latency than EBS Overkill for single-instance use Only for Linux (no Windows support)

■ Cost: ~\$0.30/GB/month for Standard. Infrequent Access tier ~\$0.025/GB. ~3-4x more expensive than EBS gp3

RDS — Relational Database Service

- Managed relational DB — MySQL, PostgreSQL, Oracle, SQL Server, MariaDB, Aurora
- **Multi-AZ** — synchronous standby replica in another AZ for automatic failover (~1-2 min)
- **Read Replicas** — async replicas for read scaling; can be cross-region
- **Aurora** — AWS-native, MySQL/Postgres compatible, up to 5x faster, auto-scales storage

Best Case

- ACID transactions, complex queries, joins — e-commerce, banking, ERP systems
- Example: E-commerce: orders, inventory, payments with complex JOINS and transactions — RDS PostgreSQL Multi-AZ

Pros & Cons

Pros	Cons
Fully managed: patching, backups, failover Familiar SQL, supports multiple engines Multi-AZ for high availability Read Replicas for read scaling	Vertical scaling only (no horizontal write scaling) Expensive for large instances Schema migrations are painful at scale Max storage 64TB (Aurora: 128TB)

■ Cost: Per DB instance hour + storage GB/month + I/O requests + backup storage. db.t3.medium ~\$0.068/hr. Multi-AZ doubles instance cost. Aurora ~20% more than RDS but better performance

DynamoDB — Managed NoSQL Database

- Key-value and document NoSQL DB — single-digit millisecond latency at any scale
- **Primary key:** Partition key alone, or Partition key + Sort key (composite)
- **GSI/LSI** — Global/Local Secondary Indexes for alternate query patterns
- **DynamoDB Streams** — capture item changes for event-driven processing
- **Capacity modes:** On-demand (pay per request) or Provisioned (set RCU/WCU)

Best Case

- High-throughput key-value access at massive scale — session store, shopping cart, leaderboards
- Example: Session store for 10M users — each session fetched by sessionId in <5ms; auto-scales with zero downtime

Pros & Cons

Pros	Cons
Single-digit ms latency at any scale Serverless option (on-demand pricing) Auto-scaling, no schema management DynamoDB Streams for event-driven arch	No joins — must design around access patterns Limited query flexibility (no ad-hoc SQL) Can get expensive for read-heavy large items Learning curve for data modeling

■ Cost: On-demand: \$1.25 per million writes, \$0.25 per million reads. Provisioned: per RCU/WCU/hour. Storage \$0.25/GB/month. Free tier: 25GB + 25 RCU/WCU

3. Networking

VPC — Virtual Private Cloud

- Logically isolated network within AWS — your own private datacenter in the cloud
- **Subnets:** Public (has internet gateway route), Private (no direct internet access)
- **Security Groups** — stateful, instance-level firewall (allow rules only)
- **NACLs** — stateless, subnet-level firewall (allow and deny rules)
- **NAT Gateway** — allows private subnet instances to reach internet without being reachable from it
- **VPC Peering / Transit Gateway** — connect VPCs to each other

Best Case

- Isolating resources in a private network with controlled, layered access
- Example: Public subnet: ALB (internet-facing). Private subnet: EC2 app servers. Isolated subnet: RDS. Nothing in private/isolated subnets is reachable from the internet

Pros & Cons

Pros	Cons
Full network control and security isolation Flexible routing and IP addressing Default VPC ready to use Supports hybrid cloud with VPN/Direct Connect	NAT Gateway adds significant cost VPC peering doesn't scale well (use Transit Gateway) Complex setup for multi-account architectures Misconfiguration can expose resources

■ Cost: VPC itself is free. NAT Gateway: ~\$0.045/hr + \$0.045/GB processed (can be expensive at scale). VPC peering: data transfer charges apply

Load Balancers — ALB / NLB

- **ALB (Application LB)** — Layer 7, HTTP/HTTPS, path/host-based routing, WebSocket support
- **NLB (Network LB)** — Layer 4, TCP/UDP/TLS, ultra-low latency, static IP, millions of req/sec
- **CLB (Classic LB)** — Legacy, avoid for new deployments

ALB Best Case

- HTTP/HTTPS traffic routing to multiple services with path or host-based rules
- Example: Route `/api/*` to backend EC2 ASG, `/admin/*` to separate admin service, `/static/*` to S3 — all on one ALB

NLB Best Case

- Ultra-low latency TCP/UDP, static IP requirement, non-HTTP protocols
- Example: Real-time gaming server needing static IP and TCP connections with sub-millisecond latency

Pros & Cons

	ALB	NLB
Pros	Layer 7 routing, WebSocket, health checks, WAF integration	Ultra-low latency, static IP, handles millions of req/sec, TCP/UDP
Cons	Higher cost per LCU for high throughput, no static IP	No Layer 7 features — no path routing, no header inspection

■ Cost: ALB/NLB: per hour (~\$0.008/hr) + per LCU consumed. LCU based on connections, bandwidth, requests. Typically \$15-50/month for moderate traffic

CloudFront — CDN

- Content Delivery Network — 400+ edge locations globally cache and serve content
- **Origins:** S3, ALB, EC2, API Gateway, any HTTP server
- **Cache behaviors** — configure TTL, query string forwarding, headers per path pattern
- Built-in **DDoS protection** via AWS Shield Standard (free)

Best Case

- Serving static assets or cacheable API responses globally with low latency
- Example: SPA frontend (JS/CSS/images) in S3, served via CloudFront from edge near user — 50ms instead of 500ms

Pros & Cons

Pros	Cons
Reduces origin load dramatically Global low latency from edge locations DDoS protection included HTTPS with free SSL cert (ACM)	Cache invalidation can be slow/tricky Data transfer out costs add up at scale Debugging cache issues can be frustrating Not suitable for real-time, non-cacheable content

■ Cost: Per GB transferred out from edge (~\$0.085/GB first 10TB) + per 10K HTTPS requests (~\$0.01). Much cheaper than serving from origin for high traffic

Route 53 — DNS Service

- Highly available DNS service with advanced routing policies
- **Routing policies:** Simple, Weighted, Latency-based, Failover, Geolocation, Multivalue
- **Health checks** — monitor endpoints; trigger failover automatically

Best Case

- DNS with health-check-based failover and latency-based routing across regions
- Example: Route users to us-east-1 or eu-west-1 based on latency; auto-failover to backup region if primary health check fails

Pros & Cons

Pros	Cons
Highly available (100% SLA) Advanced routing (weighted, latency, geo) Integrates with AWS health checks Domain registration available	TTL propagation delay on changes Complex routing policies need careful planning Health check costs add up with many endpoints

■ Cost: \$0.50/month per hosted zone + \$0.40 per million queries. Health checks: \$0.50-\$0.75/month each. Domain registration: varies by TLD

4. Messaging & Eventing

SQS — Simple Queue Service

- Fully managed message queue — decouple producers and consumers
- **Standard Queue** — at-least-once delivery, best-effort ordering, near-unlimited throughput
- **FIFO Queue** — exactly-once processing, strict ordering, 3000 msg/sec
- **Visibility timeout** — message hidden after receive; if not deleted in time, reappears for retry
- **DLQ (Dead Letter Queue)** — failed messages moved here after N retries for inspection
- **Long polling** — waits up to 20s for messages (reduces empty response costs)

Best Case

- Decoupling producer/consumer with guaranteed delivery; absorbing traffic spikes without data loss
- Example: Order placed → SQS → Lambda processes order async. If Lambda fails, message reappears. Spike of 10K orders handled without dropping any

Pros & Cons

Pros	Cons
Fully managed, scales automatically DLQ for failed message handling Visibility timeout enables safe retry Decouples services — absorbs spikes	Not a streaming platform (no message replay) 256KB max message size Polling-based adds slight latency FIFO limited to 3000 msg/sec

■ Cost: First 1M requests/month free. Then \$0.40 per million (Standard) or \$0.50 per million (FIFO). Very cheap for most workloads

SNS — Simple Notification Service

- Fully managed pub/sub — push messages to multiple subscribers simultaneously
- **Subscribers:** SQS, Lambda, HTTP/S endpoints, email, SMS, mobile push
- **Fan-out pattern** — one SNS topic → multiple SQS queues → parallel processing
- Message filtering — subscribers receive only messages matching their filter policy

Best Case

- One event needs to trigger multiple different systems simultaneously
- Example: Payment completed → SNS topic → simultaneously triggers: email Lambda, inventory SQS, analytics SQS, mobile push notification

Pros & Cons

Pros	Cons
Push-based — instant delivery Fan-out to many subscribers at once Multiple subscriber types Message filtering reduces processing	No message persistence (fire and forget) No replay capability 256KB max message size Not suitable for ordered processing

■ Cost: First 1M publishes/month free. Then \$0.50 per million. SMS and email have separate per-message costs. Very affordable

EventBridge — Event Bus

- Serverless event bus — route events from AWS services, SaaS apps, and custom apps
- **Rules** — filter events by pattern and route to targets (Lambda, SQS, Step Functions, etc.)
- **Schema Registry** — auto-discovers and stores event schemas
- Integrates with 100+ AWS services and SaaS (Stripe, Zendesk, Datadog)

Best Case

- Event-driven architecture with rich routing rules across AWS services and SaaS integrations
- Example: EC2 instance stops unexpectedly → EventBridge rule matches → triggers Lambda to alert ops team on Slack and create PagerDuty incident

Pros & Cons

Pros	Cons
Rich event filtering and routing 100+ native AWS service integrations Schema registry for event discovery No infrastructure to manage	More complex than SNS for simple use cases Slightly higher latency than SQS/SNS Newer service — fewer community resources Debugging event routing can be tricky
■ Cost: \$1.00 per million events published (custom/third-party). AWS service events free. Very cost-effective for event-driven architectures	

SQS vs SNS vs EventBridge — When to Use Which

	SQS	SNS	EventBridge
Pattern	Queue (pull)	Pub/Sub (push)	Event routing
Delivery	At-least-once	At-least-once	At-least-once
Ordering	FIFO option	No ordering	No ordering
Replay	No	No	Archive + replay
Use when	Decouple, absorb spikes, async processing	Fan-out to multiple subscribers	Complex routing, SaaS integration, AWS event reactions

5. Security & IAM

IAM — Identity & Access Management

- **Users** — individual people with long-term credentials (avoid for apps — use roles)
- **Groups** — collection of users; assign permissions to group
- **Roles** — temporary credentials assumed by AWS services, apps, or users (best practice)
- **Policies** — JSON documents defining Allow/Deny on resources and actions
- **Least privilege principle** — grant only the minimum permissions needed
- **IAM vs Resource policies** — IAM attached to identity; Resource policy attached to resource (e.g., S3 bucket policy)

Best Case

- Controlling which AWS services/users can access which resources — core of all AWS security
- Example: EC2 needs to read from S3 → assign IAM Role to EC2 with S3 read-only policy. No hardcoded credentials, auto-rotated temp creds

Pros & Cons

Pros	Cons
------	------

Fine-grained permissions No credential management with roles Free — no additional cost Audit via CloudTrail

Complex policies are error-prone Overly permissive policies = major security risk Policy evaluation logic can be confusing Role chaining adds complexity

■ Cost: IAM is completely free

KMS — Key Management Service

- Managed encryption key service — create, control, and audit cryptographic keys
- **CMK (Customer Managed Key)** — you control rotation, deletion, usage policies
- **AWS Managed Key** — AWS manages for you, used automatically by services like S3, RDS
- Integrates with S3, RDS, EBS, Lambda, Secrets Manager for at-rest encryption
- All key usage logged in CloudTrail — full audit trail

Best Case

- Centralized management of encryption keys for data at rest across all AWS services
- Example: RDS encrypted with CMK; S3 SSE-KMS; Lambda env vars encrypted — all key usage auditable in CloudTrail

Pros & Cons

Pros	Cons
Centralized key management Automatic annual key rotation Full audit trail via CloudTrail Native integration with AWS services	\$0.03 per 10K API calls adds up at scale Key deletion has 7-30 day waiting period Custom key rotation requires manual Lambda Cross-region requires key replication

■ Cost: \$1/month per CMK + \$0.03 per 10K API calls. AWS managed keys are free. Can be significant for high-encryption-volume workloads

Secrets Manager

- Securely store and automatically rotate secrets — DB passwords, API keys, tokens
- **Auto-rotation** — rotates secrets on a schedule using Lambda; zero downtime
- Fine-grained IAM access control per secret
- Versioning — keeps previous versions during rotation

Best Case

- Storing DB credentials or API keys that need automatic rotation — production security requirement
- Example: Spring Boot fetches DB password from Secrets Manager at startup via SDK. Secrets Manager rotates it every 30 days — app always gets latest without redeployment

Pros & Cons

Pros	Cons
Auto-rotation with zero downtime Audit trail of all secret access Fine-grained IAM per secret Versioning for rollback	\$0.40/secret/month + \$0.05 per 10K API calls Overkill for non-sensitive config More expensive than Parameter Store Application needs SDK integration

■ Cost: \$0.40/secret/month + \$0.05 per 10K API calls. A small app with 10 secrets = ~\$4/month

Parameter Store (SSM)

- Hierarchical config and secrets storage — free for standard parameters
- **Standard params** — free, up to 4KB. **Advanced params** — larger size, policies, cost money
- **SecureString** — encrypted with KMS at no extra cost for standard tier
- Good for: feature flags, environment config, app settings, non-rotating secrets

Best Case

- Storing non-sensitive or mildly sensitive config values with hierarchical organization

■ Example: Store /prod/db/endpoint, /prod/app/featureFlags, /prod/app/maxConnections — free, organized, environment-specific

Pros & Cons

Pros	Cons
Free for standard parameters Hierarchical naming (/env/service/key) Integrates with ECS, Lambda, EC2 SecureString for basic encryption	No auto-rotation (need custom Lambda) Advanced params cost money Less feature-rich than Secrets Manager Not ideal for high-frequency secret rotation

■ Cost: Standard parameters: free (up to 10K). Advanced: \$0.05 per parameter/month. SecureString with KMS: standard KMS costs apply

Cognito — User Authentication

- **User Pools** — user directory with sign-up, sign-in, MFA, social login (Google, Facebook, Apple)
- **Identity Pools** — exchange Cognito tokens for AWS credentials to access AWS services directly
- Returns JWT tokens (ID token, Access token, Refresh token) — OIDC compliant
- Supports SAML and OIDC federation for enterprise SSO

Best Case

- Adding user auth with social login and MFA to mobile/web app without building auth from scratch

■ Example: Mobile app needs Google + email/password login → Cognito User Pool handles auth → returns JWT → Spring Boot backend validates JWT

Pros & Cons

Pros	Cons
Handles auth complexity out of the box Supports OAuth2/OIDC natively Scales to millions of users MFA, password policies built-in	Limited customization of auth flows Hosted UI is basic — hard to brand Can get complex with advanced federation Migrating away from Cognito is painful

■ Cost: Free up to 50K MAU. Then \$0.0055/MAU (up to 100K). SMS MFA has additional per-message cost. Affordable for most apps

6. Developer & Deployment Tools

CodePipeline + CodeBuild + CodeDeploy — AWS CI/CD

- **CodePipeline** — orchestrates the CI/CD workflow stages (Source → Build → Test → Deploy)
- **CodeBuild** — fully managed build service (compile, test, Docker build) — no Jenkins needed
- **CodeDeploy** — automated deployment to EC2, Lambda, ECS with blue/green and canary strategies

Best Case

- Fully AWS-native CI/CD without managing Jenkins or external runners

■ Example: Push to main → CodePipeline triggers → CodeBuild runs tests + builds Docker image → pushes to ECR → CodeDeploy blue/green to ECS

Pros & Cons

Pros	Cons
Native AWS integration — no config for IAM, ECR, ECS No CI/CD servers to manage Blue/green and canary deployments Pay per pipeline execution	Less flexible than GitHub Actions/Jenkins Vendor lock-in to AWS UI less intuitive than modern CI/CD tools Limited plugin ecosystem

■ Cost: CodePipeline: \$1/active pipeline/month. CodeBuild: \$0.005/build minute (varies by instance). CodeDeploy: free for EC2/Lambda; \$0.02 per on-premise update

CloudFormation / CDK — Infrastructure as Code

- **CloudFormation** — define AWS infrastructure in JSON/YAML templates
- **CDK (Cloud Development Kit)** — define infra in real code (Java, Python, TypeScript) that compiles to CloudFormation
- **Stack** — unit of deployment; rollback automatically on failure
- **Drift detection** — identifies manual changes made outside CloudFormation

Best Case

- Defining and versioning entire AWS infrastructure reproducibly across environments
- Example: CDK stack defines VPC + subnets + RDS + ECS service + ALB — deploy identical infra to dev/staging/prod with one command

Pros & Cons

Pros	Cons
Version-controlled, repeatable infra Drift detection Atomic rollback on failure CDK adds type safety and reuse	CloudFormation templates get complex fast Deployment can be slow (20-30 min for large stacks) Debugging failures can be tedious CDK still compiles to CFN — inherits its limits

■ Cost: CloudFormation is free — you pay only for resources created. CDK is open source and free

ECR — Elastic Container Registry

- Managed Docker container image registry — private repos within AWS
- Tight IAM integration — no separate registry credentials needed for ECS/EKS
- **Image scanning** — automatic vulnerability scanning on push
- **Lifecycle policies** — auto-delete old images to reduce storage costs

Best Case

- Storing Docker images for ECS/EKS deployments within AWS ecosystem
- Example: CodeBuild builds Docker image → pushes to ECR → ECS pulls image from ECR at deploy — low latency, no external registry auth

Pros & Cons

Pros	Cons
IAM-integrated — no registry credentials Fast pulls within AWS network Vulnerability scanning Lifecycle policies for cleanup	\$0.10/GB/month after 500MB free Data transfer costs for cross-region pulls Less feature-rich than Docker Hub No free public registry tier

■ Cost: \$0.10/GB/month storage. Data transfer within same region to ECS/EKS is free. 500MB free per month

7. Monitoring & Observability

CloudWatch — Metrics, Logs & Alarms

- Centralized monitoring — metrics from all AWS services automatically flow in
- **Metrics** — CPU, memory, request count, latency; 1-min default granularity (1-sec available)
- **Logs** — stream application logs; query with CloudWatch Logs Insights
- **Alarms** — trigger SNS, Auto Scaling, Lambda based on metric thresholds
- **Dashboards** — custom visualizations across services and accounts

Best Case

- Centralized metrics, logs, and alerting for all AWS resources in one place
- Example: EC2 CPU > 80% for 5 min → CloudWatch Alarm → trigger Auto Scaling + SNS alert. Spring Boot logs streamed to CloudWatch for centralized querying

Pros & Cons

Pros	Cons
Native integration with all AWS services Dashboards, alarms, log insights No agent needed for basic metrics Retention policies for log management	Log Insights queries can be slow/expensive at scale 1-min metric granularity by default Not ideal as primary APM tool Cost grows quickly with high log volume

■ Cost: Metrics: \$0.30/metric/month. Log ingestion: \$0.50/GB. Log Insights: \$0.005/GB scanned. Alarms: \$0.10/alarm/month. Free tier: 10 metrics, 5GB logs

X-Ray — Distributed Tracing

- End-to-end request tracing across microservices — find latency bottlenecks
- **Service Map** — visual graph of all services and their latencies
- **Segments and Subsegments** — trace each hop in a request (Lambda, DB, HTTP call)
- SDK instrumentation required in application code — Spring Boot via X-Ray SDK
- **Sampling rules** — only a % of requests traced by default to control cost

Best Case

- Finding where latency or errors occur in a distributed system across multiple services
- Example: API call: ALB → Lambda → DynamoDB → external API. X-Ray shows DynamoDB call taking 1.8s out of 2s total — pinpoints the bottleneck

Pros & Cons

Pros	Cons
End-to-end trace visualization Service map shows error rates and latency Native integration with Lambda, ECS, API GW Latency percentiles (p50, p90, p99)	SDK instrumentation needed in code Sampling means not all requests traced Cost grows at high request volume Limited compared to Datadog/New Relic APM

■ Cost: First 100K traces/month free. Then \$5 per million traces recorded, \$0.50 per million scanned. Manageable with sampling

CloudTrail — Audit & Compliance

- Records every AWS API call — who did what, when, from where
- **Event history** — 90 days of management events free in console
- Create trails to store events in S3 long-term; query with Athena
- Covers: Console actions, CLI commands, SDK calls, AWS service-to-service calls

Best Case

- Security auditing and compliance — who accessed or changed what in your AWS account
- Example: S3 bucket deleted unexpectedly → CloudTrail shows IAM user 'dev-john', from IP 203.x.x.x, at 14:32 UTC — instant accountability

Pros & Cons

Pros	Cons
Covers all AWS API calls automatically 90-day free event history Compliance and security forensics Integrates with CloudWatch for alerts	Not real-time — slight delay in events Long-term storage needs S3 + cost Querying at scale needs Athena Data events (S3 object reads) cost extra

■ Cost: First trail (management events) free. Additional trails: \$2 per 100K events. Data events (S3, Lambda): \$0.10 per 100K. S3 storage costs for long-term retention

8. Scaling Strategies

Vertical vs Horizontal Scaling

- **Vertical (Scale Up)** — Bigger instance (more CPU/RAM). Simple but has limits; causes downtime; single point of failure
- **Horizontal (Scale Out)** — More instances. Preferred for availability and elasticity; needs stateless design

EC2 Auto Scaling

- **Target Tracking** — maintain a metric at target value (e.g., CPU = 50%). Simplest, recommended for most cases
- **Step Scaling** — different scaling steps based on alarm breach magnitude (e.g., +2 instances at 70% CPU, +4 at 90%)
- **Scheduled Scaling** — scale at known times (e.g., add capacity every weekday 8am, reduce at 8pm)
- **Predictive Scaling** — ML-based, forecasts demand and scales proactively
- Always use **ALB + ASG** pattern — ALB distributes traffic, ASG manages capacity

Lambda Scaling

- **Concurrency** — Lambda scales by adding concurrent executions automatically (default 1000 per account)
- **Reserved Concurrency** — guarantee N executions for a function; also caps maximum concurrency
- **Provisioned Concurrency** — pre-warm instances to eliminate cold starts for latency-sensitive functions

Database Scaling

- **RDS Read Replicas** — offload read traffic; up to 5 replicas; async replication (slight lag)
- **RDS Multi-AZ** — HA, not for read scaling; synchronous standby for failover
- **Aurora Auto Scaling** — automatically adds read replicas based on load
- **DynamoDB On-Demand** — instant scaling with no capacity planning; pay per request
- **DynamoDB Auto Scaling** — adjust provisioned RCU/WCU based on utilization targets

Caching for Scale

- **ElastiCache Redis** — cache DB query results, session data, leaderboards. Reduces DB load dramatically
- **ElastiCache Memcached** — simpler, multi-threaded caching for high-throughput use cases
- **CloudFront** — cache at edge; offloads origin for static and semi-static content
- **DAX** — DynamoDB Accelerator; in-memory cache for DynamoDB; microsecond latency

Queue-based Scaling

- Use **SQS queue depth** as a CloudWatch metric to trigger EC2/ECS/Lambda scaling
 - Consumers scale based on number of messages in queue — decouples scaling of producers and consumers
- Example: During flash sale: orders spike → SQS queue grows → CloudWatch alarm → ASG adds EC2 workers → queue drains

Cost-aware Scaling

- **Spot Instances** — up to 90% cheaper than On-Demand; best for fault-tolerant, stateless workloads
- **Spot + On-Demand mix** — ASG with mixed instances policy: Spot for base, On-Demand for critical
- **Savings Plans / Reserved Instances** — commit 1-3 years for 40-60% savings on predictable baseline
- **Scale to zero** — Lambda and Fargate: no traffic = no cost; EC2 ASG minimum can be 0 for dev environments

9. Frequently Asked & Tricky Interview Questions

Compute

Q1. EC2 vs Lambda vs ECS — when to use which?

→ EC2: full control, long-running. Lambda: event-driven, short, spiky. ECS: containerized microservices. Lambda cheapest for spiky; EC2 for steady sustained load

Q2. What is a cold start in Lambda? How do you fix it?

→ First invocation spins up container — adds 100ms-1s latency. Fix with Provisioned Concurrency (pre-warm) or keep functions warm with scheduled pings

Q3. What is an AMI?

→ Amazon Machine Image — pre-configured OS + software snapshot used to launch EC2 instances consistently

Q4. Difference between Security Groups and NACLs?

→ Security Groups: stateful, instance-level, allow rules only. NACLs: stateless, subnet-level, allow and deny rules. SGs are more commonly used

Q5. What are EC2 Spot Instances?

→ Spare EC2 capacity at up to 90% discount. Can be interrupted with 2-min notice. Best for fault-tolerant, stateless, batch workloads

Q6. What is ECS Fargate?

→ Serverless container runtime — you define task CPU/memory, AWS manages the underlying EC2 nodes. No cluster management needed

Q7. When would you use EKS over ECS?

→ EKS when you need full Kubernetes ecosystem, service mesh (Istio), complex multi-cluster setup, or portability across clouds

Storage

Q8. S3 vs EBS vs EFS — when to use which?

→ S3: object storage, unstructured data, backups. EBS: block storage for single EC2 (databases). EFS: shared filesystem for multiple EC2s simultaneously

Q9. What is an S3 presigned URL?

→ Time-limited URL allowing upload/download directly to/from S3 without exposing AWS credentials. Generated server-side, used client-side

Q10. What is S3 versioning?

→ Keeps all versions of an object. Protects against accidental deletes and overwrites. Increases storage cost

Q11. RDS vs DynamoDB — how do you choose?

→ RDS: relational data, complex queries, joins, ACID transactions. DynamoDB: high-throughput key-value, known access patterns, massive scale

Q12. What is RDS Multi-AZ?

→ Synchronous standby replica in another AZ. Automatic failover in 1-2 min. For HA — NOT for read scaling (use Read Replicas for that)

Q13. What is DynamoDB GSI?

→ Global Secondary Index — allows querying DynamoDB on non-primary-key attributes. Must be planned at design time

Q14. What are S3 storage classes?

→ Standard (frequent access), Intelligent-Tiering (auto moves between tiers), Standard-IA (infrequent), Glacier (archival minutes), Glacier Deep Archive (hours, cheapest)

Networking

Q15. What is a VPC?

→ Virtual Private Cloud — logically isolated network in AWS. Contains subnets, route tables, internet gateways, security groups

Q16. Public vs Private subnet?

→ Public subnet has route to Internet Gateway — resources can reach/be reached from internet. Private subnet has no IGW route — needs NAT Gateway for outbound

Q17. Security Group vs NACL?

→ SG: stateful (return traffic auto-allowed), instance level, allow only. NACL: stateless (must allow both directions), subnet level, allow and deny

Q18. What is a NAT Gateway?

→ Allows private subnet instances to initiate outbound internet connections without being reachable from internet. Managed, highly available, expensive per GB

Q19. ALB vs NLB — when to use which?

→ ALB: HTTP/S, path/host routing, WebSocket, Layer 7. NLB: TCP/UDP, ultra-low latency, static IP, Layer 4. Use ALB for web apps, NLB for non-HTTP or extreme performance

Q20. What is CloudFront?

→ CDN with 400+ edge locations. Caches content closer to users. Reduces latency, origin load, and bandwidth costs. Includes DDoS protection

Q21. What is Route 53 latency-based routing?

→ Routes users to the AWS region with lowest latency for that user. e.g., EU users go to eu-west-1, US users to us-east-1 automatically

Messaging

Q22. SQS vs SNS — difference?

→ SQS: queue, pull-based, one consumer per message, persists until consumed. SNS: pub/sub, push-based, fan-out to many subscribers simultaneously

Q23. What is SQS visibility timeout?

→ After a consumer receives a message, it becomes invisible for the timeout duration. If not deleted in time, it reappears for another consumer to retry

Q24. What is a Dead Letter Queue?

→ SQS queue where messages are sent after N failed processing attempts. Used to isolate and investigate failed messages without blocking the main queue

Q25. What is the SQS fan-out pattern?

→ SNS topic → multiple SQS queues. One event triggers multiple independent consumers in parallel. Used for order processing, notifications, analytics simultaneously

Q26. SQS Standard vs FIFO?

→ Standard: unlimited throughput, at-least-once, best-effort order. FIFO: exactly-once, strict order, 3000 msg/sec. Use FIFO when order matters

Q27. What is EventBridge?

→ Serverless event bus that routes events between AWS services, SaaS apps, and custom apps using rules and filters. More powerful routing than SNS

Security

Q28. What is an IAM Role?

→ Identity with temporary credentials that can be assumed by AWS services, Lambda, EC2, etc. Best practice over hardcoded keys. Auto-rotated credentials

Q29. IAM Role vs IAM User?

→ User: long-term credentials for humans. Role: temporary credentials for services/apps. Always use roles for AWS services — never hardcode access keys

Q30. What is the least privilege principle?

→ Grant only the minimum permissions needed to perform a task. Start with no access and add only what's required

Q31. Secrets Manager vs Parameter Store?

→ Secrets Manager: auto-rotation, more features, costs \$0.40/secret/month. Parameter Store: free for standard, no auto-rotation. Use SM for rotating DB creds, PS for config

Q32. What is KMS?

→ Managed encryption key service. Create and control keys used to encrypt data at rest in S3, RDS, EBS, Lambda. All usage logged in CloudTrail

Q33. What is Cognito User Pool vs Identity Pool?

→ User Pool: user directory, handles sign-up/sign-in, returns JWT. Identity Pool: exchanges tokens for temporary AWS credentials to access AWS services directly

Q34. How does IAM policy evaluation work?

→ Default deny → explicit allow wins → explicit deny always wins. If no policy matches, access is denied. Deny takes precedence over any allow

Scaling & Architecture

Q35. Difference between horizontal and vertical scaling?

→ Vertical: bigger instance (scale up) — has limits, causes downtime. Horizontal: more instances (scale out) — preferred, needs stateless design

Q36. What is an Auto Scaling Group?

→ EC2 feature that automatically adds/removes instances based on policies. Always pair with ALB for load distribution

Q37. What scaling policies does ASG support?

→ Target Tracking (maintain a metric), Step Scaling (different steps per threshold), Scheduled (time-based), Predictive (ML-based forecasting)

Q38. How do you scale a database in AWS?

→ RDS Read Replicas for reads, Multi-AZ for HA. Aurora auto-scales replicas. DynamoDB on-demand for instant scaling. ElastiCache to reduce DB pressure

Q39. What is ElastiCache used for?

→ In-memory cache (Redis or Memcached) to reduce DB load. Cache frequent queries, sessions, leaderboards. Sub-millisecond latency

Q40. How do you use SQS for scaling?

→ Use SQS queue depth as CloudWatch metric → alarm when depth grows → trigger ASG to add consumers. Decouples producer and consumer scaling

Q41. What are Spot Instances best for?

→ Fault-tolerant, stateless, batch workloads that can handle interruption — e.g., data processing, CI/CD builds, rendering. Not for databases or stateful apps

Q42. What is Reserved Instance vs Savings Plan?

→ Both give 40-60% discount for commitment. RI: specific instance type/region. Savings Plan: more flexible — applies to any instance type/region/Lambda

Monitoring & DevOps

Q43. What is CloudWatch used for?

→ Metrics, logs, alarms, dashboards for AWS resources. Triggers Auto Scaling, SNS alerts. Central observability for all AWS services

Q44. CloudWatch vs X-Ray?

→ CloudWatch: metrics, logs, alarms — what is happening. X-Ray: distributed tracing — where in the request chain is it happening and why

Q45. What is CloudTrail?

→ Audit log of all AWS API calls — who did what, when, from where. Essential for security forensics and compliance

Q46. What is Infrastructure as Code (IaC)?

→ Define infrastructure in code (CloudFormation/CDK) — version-controlled, repeatable, auditable deployments. Eliminates manual console clicks

Q47. CloudFormation vs CDK?

→ CloudFormation: JSON/YAML templates, verbose. CDK: write in Java/Python/TypeScript, compiles to CloudFormation. CDK preferred for complex infra

Q48. What is blue/green deployment?

→ Run two identical environments (blue=current, green=new). Switch traffic to green once tested. Instant rollback by switching back. Zero downtime

Q49. What is canary deployment?

→ Gradually shift traffic to new version — e.g., 5% → 25% → 100%. Monitor for errors at each step. Reduces blast radius of bad deployments

Q50. How would you design a highly available Spring Boot app on AWS?

→ Multi-AZ deployment: ALB → EC2 ASG across 2+ AZs → RDS Multi-AZ + Read Replica → ElastiCache → S3 for assets → CloudFront → Route 53 health checks + failover